DATA/CS 172: Python for Data Analysis

Intro to Dictionaries

Prof. Travis Mandel

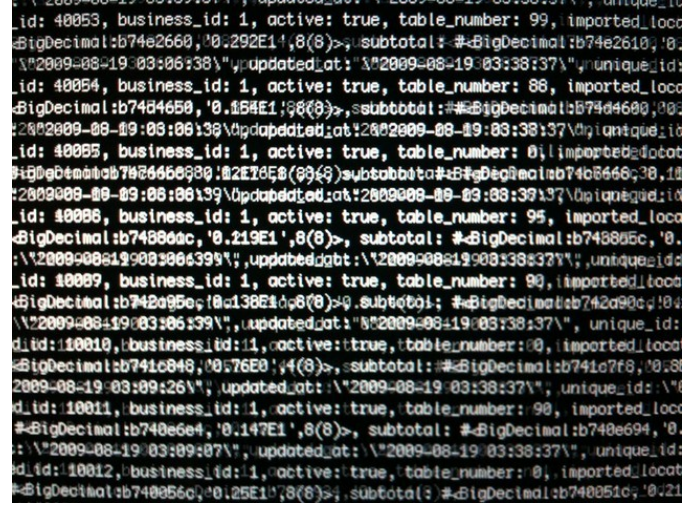
10/7/2024

UNIVERSITY of HAWAI'I®
HILO

Announcements

- Program 1 due Friday

Program 1

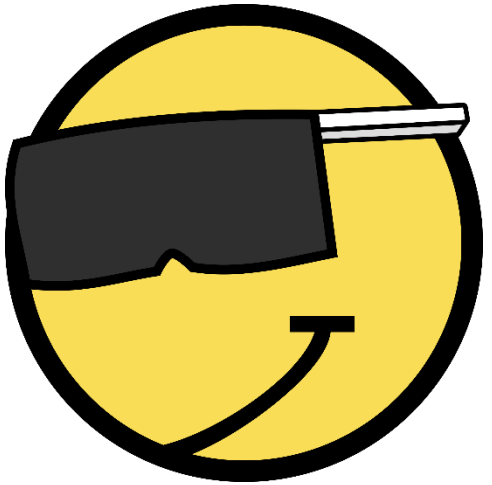


- You processed a lot of data
  - Hopefully, you have started on this by now!
- Problem: had to use lists
- You didn’t need all the data – you were interested in only a few target cuisines
  - Once things are read into lists, go through each target cuisine, count up, and print avg
  - What if I had required you to keep track of **all cuisines** -
    - Would have been a huge pain!
  - …. Is there a better way to organize things like this?

Lots of variables

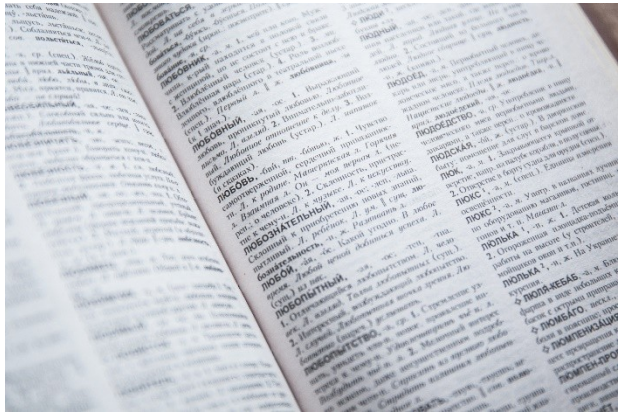
- If we have separate variables for each cuisine, it would be great
- Problem: No idea how many cuisines or what they are
  - Don’t want to hardcode
- Idea: Imagine a dictionary…
  - .. You look up a word (say “debate” or “positive test result”)
    - … or a cuisine
  - … it gives you back information about that phrase
    (or sequence)
    - For example, the count of occurrences

Also: Tuples are pretty cool



- But no way to encode name of each element
- Have to remember what order things come in
  - myTuple = (“john”, 96, “senior”)
  - …
  - (name, standing, grade) = myTuple
  - Grade is senior?!
- Not mutable (cannot easily add new field)
- Very cumbersome if need to store lots of stuff (e.g. more than 5)

Idea..



- What if we have something that maps from name to value?
  - That dictionary idea again!
- Can use it to name a tuple…
- Can use it to remember various aspects of the data
  - Useful for tons of stuff!
  - Mondays's lab will use it for unit conversion, for instance.

Dict

- Short for dictionary
- Maps from **keys** to **values**
  - Example: opposite colors
    - Red →Green
    - Blue → Orange
    - Yellow →Purple
  - Red, blue, yellow are the **keys**
  - Green, orange, purple are the **values**
- Each key maps to just one value

Initializing a Dict

- Empty dict
  - myDict = {}
  - Note the curly braces!
    - In contrast to square for lists and parens for tuples
- Dict with some stuff in there already:

```
myDict = {
        “key1” :”value1”,
        “key2”:”value2”,
        “key3”:”value3”
}
```

Keys often strings

Values could be any type (int, float, string, etc.)

Looking up something in a dict

- myDict[“key1”]
- This is very fast!

What if the key isn’t there?

- Demo
  - KeyError

- Checking if key is present?
  - YES! Our favorite in keyword again!
    - If “key1” in myDict:

Adding to a dict

- myDict[“key1”] = value
- If key exists, this will overwrite!
- If not, will add the key

Getting all keys/values

- myDict.keys()
  - Returns a list of keys
  - Note: In my opinion, not good style to rely on exact order of the keys
    - Order was basically randomized prior to Python 3.6
- myDict.values()
  - Returns a list of values
  - Again, order not guaranteed
- len(myDict) = number of keys

Looping over dicts

- Way 1 (not the best style)

```
for key in myDict.keys():
    value = myDict[key]
    print("Key is", key, "and value is", value)
```

- Way 2 (equivalent but better style)

```
for key in myDict:
    value = myDict[key]
    print("Key is", key, "and value is", value)
```

- As before, in my opinion, not good style to rely on exact order

Looping over dicts: caution

- What is the problem here?

```
desiredKey = “something”
for key in myDict:
    if key == desiredKey:
        value = myDict[key]
        print(“Found value”, value)
```

- Better:
- If desiredKey in myDict:
- value = myDict[desiredKey]

Functions on dicts you should NOT call!!

- myDict.update({key:value})
  - Should just use myDict[key] = value

- myDict.items()
  - Instead of “for (key,value) in myDict.items()”, it’s better to do:
  - for key in myDict:
    value = myDict[key]
  - items() creates an unnecessary intermediate step and can lead to bugs

Lab out

- Simple practice using dicts to convert measurements
- Will see more complex example on Wednesday